Protocolo ADS (Automation Device Specification) — Especificación común

Esta especificación define el formato de cable del protocolo ADS de Beckhoff que

deben implementar de forma idéntica las tres piezas del repositorio:

- `LibPhithon/` → cliente ADS en **Python**
- `LibC#/` → cliente ADS en **C# / .NET**
- `simulador_flutter/` → **servidor** ADS (simula un PLC TwinCAT) en **Flutter/Dart**

Cualquier ambigüedad se resuelve a favor de esta especificación. Los tres módulos

usan **little-endian** para todos los campos enteros.

1. Transporte AMS/TCP

ADS va sobre TCP. Cada mensaje comienza con una cabecera AMS/TCP de **6 bytes**:

Offset	Tamaño	Campo	Valor
0	2	Reservado	`0x0000`
2	4	`length`	uint32 LE = tamaño (bytes) de la cabecera AMS + datos (siempre ≥ 32)

Después van la **cabecera AMS (32 bytes)** y, opcionalmente, los **datos**.

```
+--------+--------+----------------------------+-------------+
| res(2) | len(4) | cabecera AMS (32 bytes)     | datos (n)   |
+--------+--------+----------------------------+-------------+
```

2. Cabecera AMS (32 bytes)

Offset	Tamaño	Campo	Tipo
0	6	`target_netid`	AmsNetId (6 bytes)
6	2	`target_port`	uint16 LE
8	6	`source_netid`	AmsNetId
14	2	`source_port`	uint16 LE
16	2	`command_id`	uint16 LE
18	2	`state_flags`	uint16 LE
20	4	`length`	uint32 LE = bytes de datos tras la

			cabecera
24	4	`error_code`	uint32 LE
28	4	`invoke_id`	uint32 LE

- `state_flags`: 0 = solicitud, 1 = respuesta, 4 = notificación ADS. En la práctica

el cliente envía `0` y el servidor responde con `1` (y `4` para notificaciones).

- `invoke_id`: identificador libre de transacción; la respuesta debe devolver el mismo.

Regla de respuesta: el servidor construye la cabecera de respuesta intercambiando

`target_netid/port` ↔ `source_netid/port` de la solicitud, manteniendo `command_id`

e `invoke_id`, y poniendo `state_flags = 1` (o `4` en notificaciones no solicitadas).

AmsNetId

6 bytes, escritos por convención como `a.b.c.d.e.f` (ej. `192.168.0.1.1.1`).

Cada byte es un octeto (0–255). La red local se representa como `127.0.0.1.1.1`.

Un `AmsNetId` vacío/inválido se serializa como `0.0.0.0.0.0`.

Puertos AMS habituales

Puerto	**Servicio**
100	Logger
300	Event logger
400	IO
500	NC
800	PLC Runtime (TC2)
851	PLC Runtime 1 (TC3)
852	PLC Runtime 2
853	PLC Runtime 3
10000	System Service

3. Comandos ADS (command_id)

Valor	**Nombre**	**Descripción**
0x0001	`READ_DEVICE_INFO`	Leer información del dispositivo
0x0002	`READ`	Leer datos (por índice/offset o por nombre)
0x0003	`WRITE`	Escribir datos
0x0004	`READ_STATE`	Leer estado del dispositivo
0x0005	`WRITE_CONTROL`	Cambiar estado del dispositivo

0x0006	ADD_DEVICE_NOTIFICATION	Suscribir notificación
0x0007	DEL_DEVICE_NOTIFICATION	Cancelar suscripción
0x0008	DEVICE_NOTIFICATION	(servidor→cliente) entrega de notificación
0x0009	READ_WRITE	Leer y escribir en una sola transacción

4. Grupos de índice (index_group)

Los comandos READ, WRITE y READ_WRITE llevan como primeros 8 bytes de datos

el index_group (uint32) y el index_offset (uint32).

Valor	Constante	Uso
0xF000	SYMTAB	Tabla de símbolos (lista)
0xF001	SYMNAME	Nombre de símbolo
0xF002	SYMVAL	Valor por (nombre)
0xF003	SYM_HNDBYNAME	Obtener handle por nombre
0xF004	SYM_VALBYHND	Leer/escribir valor por handle
0xF005	SYM_VALBYNAME	Leer/escribir valor por nombre
0xF006	SYM_INFOBYNAME	Información de símbolo por nombre
0xF008	SYM_INFOBYNAMEEX	Info extendida por nombre
0xF009	SYM_DOWNLOAD	Descarga de símbolos
0xF014	SYM_RELEASEHND	Liberar handle de símbolo
0xF080	SUMUP_READ	Lectura múltiple (sum)
0xF081	SUMUP_WRITE	Escritura múltiple (sum)
0xF082	SUMUP_READWRITE	Lectura+escritura múltiple
0xF083	SUMUP_READEX	Lectura múltiple extendida
0xF100	DEVICE_DATA	Datos de dispositivo (por offset)
0xF020	IOIMAGE_RWIB	Imagen de proceso — entradas
0xF030	IOIMAGE_RWOB	Imagen de proceso — salidas

5. Estados del dispositivo

Valor	Nombre
0	INVALID
1	IDLE
2	RESET
3	INIT
4	START

5	RUN
6	STOP
7	SAVECFG
8	LOADCFG
9	POWERFAILURE
10	POWERGOOD
11	ERROR
12	SHUTDOWN
13	SUSPEND
14	RESUME
15	CONFIG
16	RECONFIG
17	STOPPING
18	INCOMPATIBLE
19	EXCEPTION

Modos de transmisión de notificación (`trans_mode`):

Valor	**Nombre**
0	NOTRANS
1	CLIENTCYCLE
2	CLIENTONCHA (on change)
3	SERVERCYCLE
4	SERVERONCHA (on change)

6. Formatos de datos por comando

Convención: `IG` = index_group (u32), `IO` = index_offset (u32), `L` = longitud (u32),

`E` = código de error/resultado (u32). Todo little-endian.

6.1 READ_DEVICE_INFO (0x0001)

- **Solicitud**: sin datos.
- **Respuesta** (20 bytes):

Offset	**Tamaño**	**Campo**
0	1	`major_version` (u8)
1	1	`minor_version` (u8)
2	2	`version/build` (u16)
4	16	`device_name` (ASCII, relleno con `\0`)

6.2 READ (0x0002)

- **Datos** = `IG` + `IO` + parámetros de lectura.

Lectura directa (índice/offset, ej. imagen de proceso):

```
IG(4) + IO(4) + L(4)          // L = bytes a leer
```

Lectura por nombre (`SYM_VALBYNAME` 0xF005):

```
IG(4)=0xF005 + IO(4)=0 + L(4) + name_bytes(L) + L2(4)   // L=len(name), L2=bytes a leer
```

Lectura por handle (`SYM_VALBYHND` 0xF004):

```
IG(4)=0xF004 + IO(4)=handle + L(4)                      // L = bytes a leer
```

- **Respuesta (correcta)**: cabecera AMS `error_code = 0` y datos = **datos crudos**

de longitud exactamente igual a la solicitada (sin código de resultado ni longitud).

- **Respuesta (error)**: cabecera AMS `error_code ≠ 0` y, típicamente, sin datos.

6.3 WRITE (0x0003)

Escritura directa:

```
IG(4) + IO(4) + L(4) + data(L)
```

Escritura por nombre (`SYM_VALBYNAME`):

```
IG(4)=0xF005 + IO(4)=0 + L(4) + name_bytes(L) + L2(4) + data(L2)
```

Escritura por handle (`SYM_VALBYHND`):

```
IG(4)=0xF004 + IO(4)=handle + L(4) + data(L)
```

- **Respuesta**: cabecera AMS `error_code = 0` y **sin datos**.

6.4 READ_STATE (0x0004)

- **Solicitud**: sin datos.
- **Respuesta** (4 bytes): `ads_state(2) + device_state(2)`.

6.5 WRITE_CONTROL (0x0005)

- **Solicitud**: `ads_state(2) + device_state(2) + L(4) + data(L)`.
- **Respuesta**: sin datos.

6.6 READ_WRITE (0x0009)

- **Solicitud**: `IG(4) + IO(4) + read_len(4) + write_len(4) + write_data(write_len)`.
- **Respuesta**: `E(4)=0 + read_len(4) + read_data(read_len)`.

6.7 ADD_DEVICE_NOTIFICATION (0x0006)

- **Solicitud**: `IG(4) + IO(4) + attrib(24)` donde `attrib`:

Offset	Tamaño	Campo
0	4	`cb_length` = bytes a notificar
4	4	`trans_mode`
8	4	`max_delay` (ms)

12	4	`cycle_time` (unidades de 100 ns)
16	8	reservado (`0`)

- **Respuesta**: `notification_handle(4)` (u32, valor distinto de 0).

6.8 DEL_DEVICE_NOTIFICATION (0x0007)

- **Solicitud**: `notification_handle(4)`.
- **Respuesta**: sin datos.

6.9 DEVICE_NOTIFICATION (0x0008) — servidor→cliente

- **Datos** (stamps): `L(4)` = bytes totales de stamps; seguido de, por cada stamp:

Offset	Tamaño	Campo
0	4	`notification_handle`
4	8	`timestamp` (FILETIME Windows, 100 ns desde 1601-01-01)
12	4	`sample_size`
16	n	`data` (sample_size bytes)

6.10 Sum read (SUMUP_READ 0xF080)

- **Solicitud**: concatenación de entradas `IG(4) + IO(4) + L(4)`.
- **Respuesta**: concatenación por entrada de `E(4) + L(4) + data(L)`.
- `SUMUP_WRITE` (0xF081): solicitud = entradas `IG(4)+IO(4)+L(4)+data(L)`;

respuesta = un `E(4)` por entrada.

6.11 Símbolos

`SYM_HNDBYNAME` (0xF003), con `READ`:

```
IG(4)=0xF003 + IO(4)=0 + L(4) + name_bytes(L)
```

- **Respuesta**: `handle(4)` (sin código de resultado inicial).

Liberar handle (`SYM_RELEASEHND` 0xF014), con `WRITE`:

```
IG(4)=0xF014 + IO(4)=handle + L(4)=0
```

`SYM_INFOBYNAME` (0xF006), con `READ`: igual petición que 0xF003.

- **Respuesta**: `symbol_info` directamente (sin código de resultado inicial):

Offset	Tamaño	Campo
0	4	`index_group`
4	4	`index_offset`
8	4	`size` (bytes del valor)
12	4	`data_type` (u32)

16	2	flags
18	2	name_length
20	2	type_length
22	2	comment_length
24	…	name + type + comment

SYM_INFOBYNAMEEX (0xF008): igual pero con campos name_length, type_length, comment_length como **u32**.

SYMTAB (0xF000) lectura: IG(4)=0xF000 + IO(4)=0 + L(4); respuesta = entradas `entry_len(4) + IG(4) + IO(4) + size(4) + data_type(4) + flags(2) + name_length(2) + type_length(2) + comment_length(2) + name + type + comment`.

7. Tipos de datos PLC (codificación de bytes)

Todos los valores se codifican little-endian. Tipos soportados por las tres piezas:

Tipo PLC	Bytes	Codificación
BOOL	1	0 o 1
SINT/USINT/BYTE	1	entero
INT/UINT/WORD	2	int LE
DINT/UDINT/DWORD/REAL/ TIME/TOD/DT/DATE	4	int/uint LE; REAL = IEEE-754 single
LINT/ULINT/LWORD/LREAL	8	int/uint LE; LREAL = IEEE-754 double
STRING(n)	n+1	byte 0 = longitud, luego n bytes ASCII relleno \0
WSTRING(n)	2*(n+1)	word 0 = longitud, luego n UTF-16 LE
ARRAY	n*elem	elementos contiguos (row-major)
STRUCT	Σ campos	campos empaquetados en orden de declaración

Regla de empaquetado: **sin padding automático** (packed), salvo que la librería ofrezca alineación explícita.

8. Códigos de error comunes (u32)

Código	Significado
0x00000000	Sin error
0x00000001	Error interno

0x00000006	Puerto destino no encontrado
0x00000007	Máquina destino no encontrada
0x00000008	Comando desconocido
0x0000000E	Longitud AMS inválida
0x0000000F	AmsNetId inválido
0x00000012	Puerto deshabilitado
0x00000013	Puerto ya conectado
0x0000001B	Host inalcanzable
0x00000701	Servicio no soportado
0x00000702	Grupo de índice inválido
0x00000703	Offset de índice inválido
0x00000704	Lectura/escritura no permitida
0x00000705	Tamaño de parámetro incorrecto
0x00000706	Valor(es) de parámetro inválido(s)
0x0000070C	No encontrado
0x00000710	Símbolo no encontrado
0x00000712	Servidor en estado inválido
0x00000713	Modo de transmisión no soportado
0x00000714	Handle de notificación inválido
0x00000716	Sin notificación suscrita
0x00000745	Timeout agotado
0x00000750	Timeout de cliente inválido

9. Variables simuladas (comunes al simulador y a los ejemplos)

El simulador Flutter expone estas variables en el runtime PLC (puerto **851**,

NetId `127.0.0.1.1.1`):

El simulador **escucha TCP en `127.0.0.1:48898`** (puerto estándar de AMS/TCP,

configurable en la UI). Los clientes conectan su socket TCP a `48898` y dirigen

sus cabeceras AMS al NetId `127.0.0.1.1.1` y puerto AMS `851`.

Símbolo	Tipo	Valor inicial
`MAIN.bStart`	BOOL	`false`
`MAIN.bStop`	BOOL	`true`
`MAIN.nCounter`	UINT	`0`
`MAIN.iLevel`	INT	`100`
`MAIN.fTemp`	REAL	`21.5`
`MAIN.dPosition`	LREAL	`1.2345`
`MAIN.sMessage`	STRING(80)	`"Hola ADS"`
`MAIN.arrValues`	ARRAY[0..3] OF INT	`[1, 2, 3, 4]`

`MAIN.nCounter` se incrementa cada 500 ms mientras el estado del dispositivo sea

RUN, para poder probar las notificaciones. El simulador debe implementar el servidor completo de esta especificación (incluyendo notificaciones).

10. Conformidad mínima exigida

1. **Framing AMS/TCP** y cabecera AMS completos (encode + decode).
2. READ_DEVICE_INFO, READ_STATE, WRITE_CONTROL.
3. READ/WRITE con grupos SYM_VALBYNAME, SYM_VALBYHND, SYM_HNDBYNAME, SYM_INFOBYNAME, SYMTAB, imagen de proceso (IOIMAGE_RWIB/IOIMAGE_RWOB).
4. READ_WRITE.
5. Sum commands SUMUP_READ y SUMUP_WRITE.
6. Notificaciones ADD/DEL/DEVICE_NOTIFICATION.
7. Codificación/decodificación de los tipos de la sección 7.
8. Mapa de códigos de error (sección 8) expuesto como excepciones.